Manual completo de creación de ORGES

Presente y futuro · Alset Studio · LISPAI-AOS

Órganos · Reglas · UI · Props · Pulse · Mind · Deploy

Guía para crear organismos (ORGES) que no son «simples apps»: estado, reglas reactivas, contratos, superficie UI con las mismas propiedades visuales en preview y producción, e inteligencia de dominio vía AlsetInspector.

Índice

- 1. Filosofía: todo es organismo
- 2. Arranque de Studio
- 3. Anatomía de un flow .aos
- 4. Órganos de dominio (kind ORGES)
- 5. Órganos UI y props visuales
- 6. Preview = Deploy (piel compartida)
- 7. Reglas ARL e inteligencia nativa
- 8. Contratos, bind y Pulse
- 9. AlsetInspector (Mind)
- 10. Propiedades en el panel (Studio)
- 11. Tests y despliegue
- 12. Plantilla maestra y ejemplos
- 13. Checklist de excelencia

1. Filosofía: todo es organismo

En Alset, la unidad no es la pantalla ni el servicio: es el **organismo**. Tiene estado, capacidades, reglas (ARL), contratos y, si es UI, una forma visual. La meta es abstraer problemas cotidianos con el mismo lenguaje — LISPAI-AOS — de forma mantenible y ágil.

- Dominio y vista son ambos orges (kind ORGES / kind UI).
- Las props visuales viven en el IR del órgano UI (no en CSS suelto distinto por entorno).
- Mind/Inspector traduce lenguaje natural → estructura ORGES.

2. Arranque de Studio

```
go run ./cmd/alset-plataforma studio --addr 127.0.0.1:5178
# Windows portable: dist/alset-studio-windows/Iniciar-Alset-Studio.bat
```

Paneles: Toolbox, editor .aos, propiedades, consola, **preview + AlsetInspector** (cerrable y redimensionable).

3. Anatomía de un flow .aos

```
(flow nombre-app
  (version "1.0.0")
  (desc "Problema que resuelve")
  (orge …)          ;; uno o más
  (assemble …)      ;; piezas LEGO
  (bind …)          ;; contratos opcionales
  (surface ui-…)    ;; superficie activa
  (test …)          ;; verificación
  (deploy web (devices mobile tablet desktop)))
```

4. Órganos de dominio (kind ORGES)

```
(orge stock
  (kind ORGES)
  (cap "stock.consultar" "stock.actualizar")
  (provides "stock.consulta" "1.0")
  (state (stock.unidades 10) (stock.alerta false))
  (rule alerta
    (when (lte stock.unidades 3))
    (then stock.alerta true)))
```

Inteligencia nativa = reglas + estado + capacidades. No es un mock: el runtime reevalúa ARL tras cada emit.

5. Órganos UI y props visuales

La UI es un orge (kind UI). Cada nodo (button, splash, column…) es un **órgano visual** con props opcionales: padding, margin, gap, radius, color, bg, width, height, fontsize, weight, opacity, border, shadow, align, justify.

```
(orge ui-main
  (kind UI)
  (ui (column
    (splash "Mi negocio")
    (gradient)
    (badge (bind stock.unidades))
    (row
      (button "+" (emit stock.inc)
        (padding "12px") (radius "16px") (bg "#F4B400"))
      (button "-" (emit stock.dec)
        (padding "12px") (radius "16px"))))))
(surface ui-main)
```

También: (props (padding 8px) (radius 12px)) en contenedores column/row.

6. Preview = Deploy (piel compartida)

Studio y app.html de producción usan el mismo **SharedUISkin** y **applyAIsetProps**. Mismas clases (.col, .row, .splash, button, .badge) y mismas props del IR. Si algo se ve distinto, fuerza Aplicar flow + redespliegue de esa instancia.

7. Reglas ARL e inteligencia nativa

- Predicados: eq, gt, gte, lt, lte.
- when → then; indeterminate opcional.
- Tras emit, el runtime actualiza estado y re-dispara reglas.

Así el organismo «siente» umbrales (stock bajo, cupos llenos, caja abierta) sin código procedural en la vista.

8. Contratos, bind y Pulse

```
(provides "stock.consulta" "1.0")   ;; en proveedor
(requires "stock.consulta" "1.0")   ;; en consumidor
(bind venta stock stock.consulta 1.0)
```

Pulse: mensaje contractual entre orges — payload mínimo, versionado, idempotente. El bind es la forma declarativa en el flow.

9. AIsetInspector (Mind)

- Panel a la derecha del preview: cerrable (× o botón Inspector) y redimensionable (splitter).
- Analiza sintaxis, contratos, surface, tests.
- Chat de dominio → genera flow completo; «Reemplazar .aos» + Aplicar.
- API: POST /api/mind/analyze y /api/mind/chat.

10. Propiedades en el panel (Studio)

- Activa «Editar layout».
- Clic en un componente del preview.
- Ajusta padding, margin, gap, radius, color, fondo, tamaño en el panel derecho.
- «Aplicar props al órgano» — se reflejan en preview y, al desplegar, en producción.

11. Tests y despliegue

```
(test stock-ini (eq stock.unidades 10))
(deploy web (devices mobile tablet desktop))
```

Probar en Studio; Desplegar producción escribe /apps/{nombre}/ con app.html de piel compartida (instancia aislada).

12. Plantilla maestra

```
(flow mi-dominio
  (version "1.0.0")
  (desc "Describe el problema cotidiano")
  (orge nucleo
    (kind ORGES)
    (cap "dominio.consultar")
    (state (dominio.listo false))
    (rule r1 (when (eq dominio.listo false)) (then dominio.puede true)))
  (orge ui-main
    (kind UI)
    (ui (column
      (splash "Dominio")
      (gradient)
      (button "Activar" (emit dominio.listo=true)
        (padding "12px") (radius "12px")))))
  (assemble nucleo ui-main)
  (surface ui-main)
  (test ini (eq dominio.listo false))
  (deploy web (devices mobile tablet desktop)))
```

Ejemplos de dominio

- Cafetería: stock + venta + alerta umbral.
- Pedidos en ruta: estados nuevo→confirmado→ruta→entregado.
- Citas: cupos y cierre automático.
- Caja: abierta/cerrada y monto.

13. Checklist de excelencia

- Flow con desc clara del problema real.
- Al menos un orge de dominio con regla ARL.
- UI-orge con surface; props visuales en el IR.
- Preview y deploy visualmente alineados.
- Tests del estado inicial.
- Inspector sin diagnósticos críticos.
- Deploy en instancia propia /apps/nombre.